

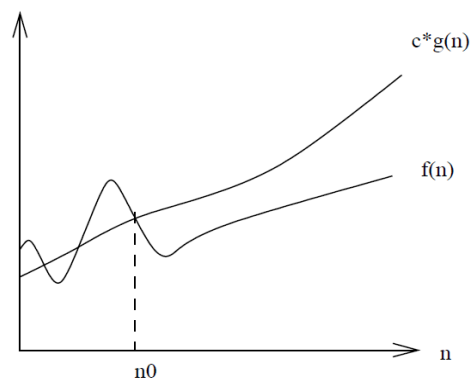
CSCI 423

Introduction to Algorithms

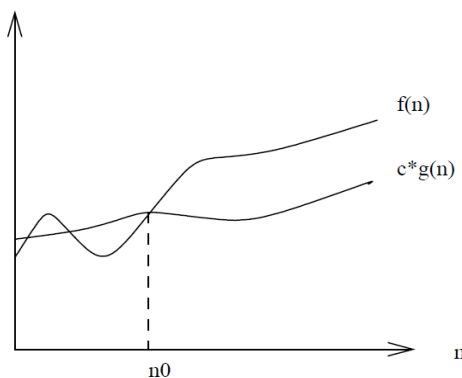


DR. RAAFAT ELFOULY

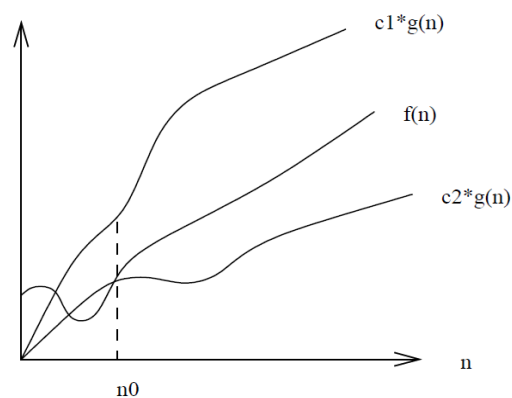
O , Ω , and Θ



(a)



(b)



(c)

The definitions imply a constant n_0 *beyond which* they are satisfied. We do not care about small values of n .

Useful summation formulas and rules

3

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

In particular, $\sum_{1 \leq i \leq n} 1 = n - 1 + 1 = n \in \Theta(n)$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \cdots + n = n(n+1)/2 \approx n^2/2 \in \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \cdots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3 \in \Theta(n^3)$$

$$\sum_{1 \leq i \leq n} i^3 = 1^3 + 2^3 + \cdots + n^3 = n^2(n+1)(n+1)/4 \approx n^4/4 \in \Theta(n^4)$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \cdots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

In particular, $\sum_{0 \leq i \leq n} 2^i = 2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1 \in \Theta(2^n)$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$

Advanced Dominance Rankings

Additional functions arise in more sophisticated analysis than we will do in this course:

$$n! \gg c^n \gg n^3 \gg n^2 \gg n^{1+\epsilon} \gg n \log n \gg n \gg \sqrt{n} \gg \log^2 n \gg \log n \gg \log n / \log \log n \gg \log \log n \gg \alpha(n) \gg 1$$

Approximating Recurrence Relations

5

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

Chapter 3



SEARCHING AND SELECTION ALGORITHMS

Chapter Outline

7

- Sequential search
- Binary search
- List Selection

Selection

8

- We want to find a particular element of an unsorted list
- Typically, this is expressed as wanting to find the K^{th} largest element
- We could find this by first sorting the entire list, but we will see that is more work than we need to do ($O(n \log n)$) so we hope to find better algorithm!!

First Selection Algorithm

9

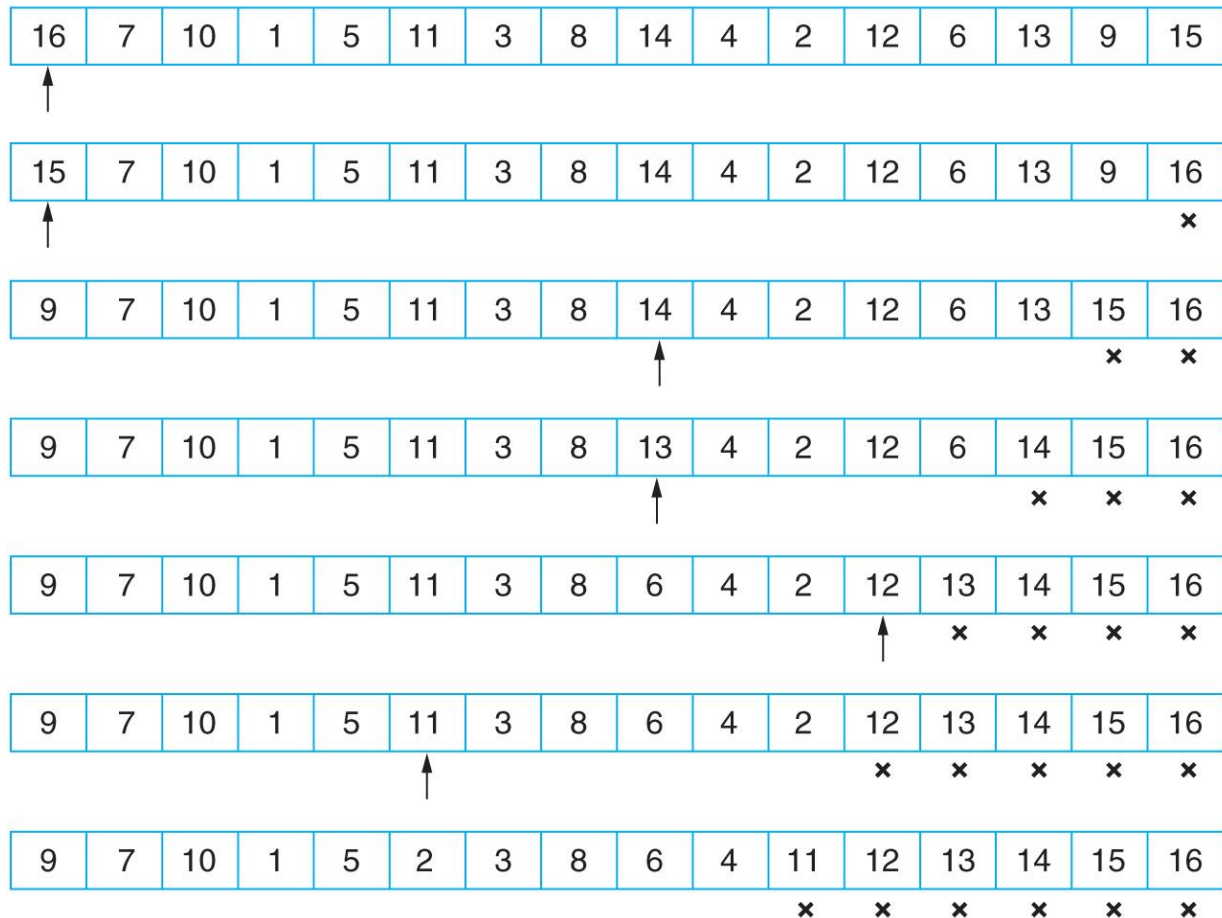
- To find the K^{th} largest element, we could move the K largest elements to the end of the list and then the K^{th} largest will be in location $\text{list}[K]$
- This involves finding the largest element and moving it to the end, then finding the second largest element and moving it to the end and so on

First Selection Algorithm Example

10

■ **FIGURE 3.4**

Finding the K^{th} largest value by moving large values to the end of the list



First Selection Algorithm

11

```
For i = 1 to K do
  largest = list[1]
  largestLocation = 1
  for j = 2 to N-(i-1) do
    if list[j] > largest then
      largest = list[j]
      largestLocation = j
    end if
  end for
  Swap( list[N-(i-1)], list[largestLocation] )
end for
return largest
```

First Selection Algorithm

12

```
For i = 1 to K do
  largest = list[1]
  largestLocation = 1
  for j = 2 to N-(i-1) do
    if list[j] > largest then
      largest = list[j]
      largestLocation = j
    end if
  end for
  Swap( list[N-(i-1)], list[largestLocation] )
end for
return largest
```

Complexity???

First Algorithm Analysis

13

- On the first pass, we compare the first element to the rest, doing $N - 1$ comparisons
- On the second pass, we compare the first element to the rest, doing $N - 2$ comparisons
- We need to do K passes, giving

$$\sum_{i=1}^K N - i = N * K - \frac{K * (K + 1)}{2} = O(N * K)$$

Few Announcements



- Mid term Oct 9th

HW3

Due Oct 16th

15

- Write a program to implement one of the following algorithms
 - Selection search (Kth largest element) Algorithm one or Two
 - Binary search
- Run the program for different input sizes (10K, 1Million, and 100 Millions)
- Measure the execution time using profile function and compare it to each algorithm complexity.
- Draw the execution time as function of input sample

Project 1 Search Algorithms Survey

Due Oct 28th (team project)

16

- You are required to do research on the state of the art/most recent search algorithms/Search Engines used in one of the following applications: Google, Facebook, Amazon ...)
- Steps:
 - Choose two recent papers (2017+) describing search algorithm/Engines in one of the above applications
 - Summarize your findings and compare complexity of the new search to traditional ones (linear and binary)
 - Write a program to implement one of the algorithms
- Deliverables:
 - Summary document describing your findings (3-4 pages at most). Including complexity of the new algorithms and graph it.
 - Program showing your implementation.

Second Selection Algorithm

17

- The first algorithm does more work than necessary, because we do not need to know the order of the larger elements, just that they are larger
- We now pick an element and partition the list into two parts with those larger toward the front and those smaller toward the end

Selection Problem

18

Find the k -th smallest element in a list of n numbers

- $k = 1$ or $k = n$

- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = ?

Selection Problem

19

Find the k -th smallest element in a list of n numbers

- $k = 1$ or $k = n$
- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = 8

The **median** is also the number that is halfway into the set.

To find the **median**, the data should be arranged in order from least to greatest. If there is an even number of items in the data set, then the **median** is found by taking the mean (average) of the two middlemost numbers.

Algorithms for the Selection Problem

20

The sorting-based algorithm: Sort and return the k -th element

Efficiency (if sorted by mergesort): $\Theta(n \log n)$

A faster algorithm is based on the array partitioning:

all are $\leq A[s]$		all are $\geq A[s]$
---------------------	--	---------------------

Assuming that the array is indexed from 0 to $n-1$ and s is a split position obtained by the array partitioning:

If $s = k-1$, the problem is solved;

if $s > k-1$, look for the k -th smallest element in the left part;

if $s < k-1$, look for the $(k-s)$ -th smallest element in the right part.

Note: The algorithm can simply continue until $s = k-1$.

Second Selection Algorithm

21

- If the partitioning element winds up at location $P = K$, we have found the K^{th} largest element
- If the partitioning element winds up at location $P > K$, the element we want is in the first part
- If the partitioning element winds up at location $P < K$, the element we want is in the second part

22

Diagram illustrating the step-by-step construction of a Huffman tree. The diagram shows four rows of 16 boxes each, representing the sequence of nodes at each level. Arrows indicate the merging process, and 'x' marks indicate where a node from a previous level is reused.

16	7	10	1	5	11	3	8	14	4	2	12	6	13	9	15
15	9	13	16	12	10	11	14	8	2	4	3	5	1	7	6
15	14	13	16	12	10	11	9	8	2	4	3	5	1	7	6
15	14	13	16	12	11	9	10	8	2	4	3	5	1	7	6

Second Selection Algorithm

23

```
if start < end then
  Partition( list, start, end, middle )
  if middle == K then
    return list[middle]
  else
    if K < middle then
      return KthLargestRecursive(list, start,
                                middle-1, K)
    else
      return KthLargestRecursive(list, middle+1,
                                end, K-(middle-start+1))
    end if
  end if
end if
```

Second Algorithm Analysis

24

- The partition function will do about N comparisons for a list of size N
- A simple analysis assumes that the list is divided in half each time
- This gives the comparison equation

$$N + \frac{N}{2} + \frac{N}{4} + \frac{N}{8} + \dots + 1 \approx 2N$$

Approximating Recurrence Relations

25

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$



- Always $O(N)$